

班級與個體多維學術評量統計診斷 App 之計量方法學設計藍圖

一、班級集體多學科關聯與跨次測驗等值化統計機制

在高中的教育實務中，班級導師在進行班級集體學業表現分析時，經常面臨跨學科難以直接比較、不同批次測驗難易度不一，以及學科間關聯被測量誤差稀釋等三大計量難題¹。為了解決這些問題，統計診斷 App 的底層架構必須超越傳統的百分制原始分數登記，引入經典測驗理論與項目反應理論的整合模型，建構科學化的跨次等值與跨科關聯分析機制¹。

跨學科關聯分析與信度衰減校正

評估班級各學科之間的關聯性，通常採用皮爾森積差相關係數或斯皮爾曼等級相關係數⁵。然而，在教育測驗中，學生在各學科的實得得分（Observed Scores）皆包含隨機測量誤差，這會導致計算出的學科間相關係數產生「衰減效應」，進而低估了學生學科能力之間的真實關聯²。為了精確診斷學科間的內在共變結構，系統必須利用經典測驗理論中的信度指標，對相關係數進行「信度衰減校正」（Correction for Attenuation）。

假設學科 X 與學科 Y 的實得得分相關係數為 r_{xy} ，而兩學科測驗的克隆巴赫信度係數（Cronbach's Alpha）分別為 α_x 與 α_y ²。校正後的真實能力相關係數 $r_{T_x T_y}$ 計算公式如下：

$$r_{T_x T_y} = \frac{r_{xy}}{\sqrt{\alpha_x \alpha_y}}$$

透過此項校正，導師能獲得排除測驗隨機誤差後的純粹學科能力相關矩陣。例如，若數學與物理的實得相關係數僅為 0.55，但兩者測驗信度分別為 0.70 與 0.65，經衰減校正後的真實關聯度將提升至 0.81²。這能精確揭示班級集體在數理學科底層認知能力上的高度同質性，避免因單次測驗編製品質不佳（信度低）而誤導教學決策²。

項目反應理論（IRT）與跨次測驗等值化機制

當導師試圖追蹤班級在第一、二、三次段考的學術表現變化時，直接對比原始平均分數是不合邏輯的，因為各次考試的試題難易度、題型分佈與鑑別度存在本質差異¹。為了解決「不同試卷無法直接對比」的困境，系統必須導入項目反應理論，特別是單參數的 Rasch 模型，利用其參數不變性（Parameter Invariance）特質，將不同批次的測驗結果置於同一個潛在能力尺度（Logit）上，進行 fair 且科學的對比¹。

在具體實現上，系統可採用共同被試設計（Common Persons Design）或共同錨題設計（Anchor Item Design）來進行跨次測驗的等值化（Test Equating）¹。在班級情境中，共同被試設計更為可行，因為整個班級的學生集體參與了多次不同的考試⁹。透過 Rasch 模型，將不同考試的試題難度與學生能力同時投射至一維連續統¹。透過錨定演算法，建立跨測驗的轉換函數，將後續測驗的得分等值化回歸到基線測驗（如開學複習考）的尺度¹。這不僅能精準呈現班級在某一學科的整體能力均值（Class Mean Logit）是否隨著教學推進而實質拉升，更能排除因期末試卷偏難而造成「全班退步」的假象，實現客觀的縱向監控¹。

以下為本系統底層核心評量與計量模型之統計特性、限制與互補關係之對比：

評量統計理論	分析基本單位	核心數學公式與變量	優勢與核心診斷功能	小樣本適應性與 App 優化策略
經典測驗理論 (CTT)	整份測驗與單一試題之實得表現 ¹	$X = T + E$ 難度 P_j 、點雙系列相關 r_{pbi} ²	運算需求低，直觀呈現實得分分佈、信度與項目描述統計 ¹	極佳。直接在客戶端進行矩陣運算，無收斂失敗風險 ⁴ 。
Rasch 模型 (IRT 1PL) ¹	學生潛在能力與單一項目之概率交互作用 ¹	$P(Y_{ni} = 1 \theta_i, \delta_j)$ ¹	具參數不變性，實現潛在能力與難度的同尺度（Logit）對齊與等值化 ¹	採用 Leslie Cohen 的 PROX 演算法，並限制單維度估計以確保小樣本收斂 ⁹ 。
DINA 模型 (CDM) ¹⁸	細粒度潛在認知技能/屬性向量 ²⁰	$P(Y_{ij} = 1 \alpha_i)$ ¹⁹	提供多維度、非連續性的技能掌握剖面，直接定位個別學生的知識盲區 ²⁰	限制屬性個數 $K \leq 5$ ，或導入非參數分類演算法（NPC）以克服參數過多問題 ²⁴ 。
增值評量模型	學生縱向成長	$Y_{it} = \beta Y_{i,prior}$	排除初始起點	採用滯後分數

(VAM) ²⁵	殘差與課堂教學淨效應 ²⁶	27	行為與背景干擾，客觀量化教學實質增值與個體異常偏離 ¹³	多元線性回歸，並進行貝氏收縮校正以降低隨機波動誤差 ²⁷ 。
---------------------	--------------------------	----	---	---

二、學生個體多維認知結構與細粒度技能掌握診斷

在微觀診斷層面，傳統的分數無法指導具體的補救教學，因為相同的分數可能掩蓋了完全不同的認知缺陷²¹。為此，App 必須結合項目反應理論的單參數 Rasch 模型以獲得精準的能力量尺，並整合認知診斷模型中的 DINA 模型，將學生的學術表現拆解為多個可觀測的認知屬性，進而生成精確的補救方針¹⁸。

適合高中班級規模之 Rasch PROX 估計演算法

雖然 Rasch 模型具有優良的計量學特質，但在高中的班級環境中，樣本量通常僅有 30 ~ 50 人，這使得依賴漸進大樣本性質的邊際極大似然估計（MMLE）或聯合極大似然估計（JMLE）容易產生不穩定、無法收斂，或估計標準誤過大的問題⁴。

為此，系統在底層採用了正常近似估計演算法（Normal Approximation Estimation Algorithm, PROX），該演算法在合理的正態分佈假設下，能提供閉式解，運算速度極快且絕不面臨無法收斂的困境，非常適合班級小樣本的現場診斷¹¹。

完全數據下的 PROX 估計（Non-iterative Formula）

當班級所有學生皆完成了整份測驗（無缺失值）時，可直接採用 Cohen (1979) 提出的非迭代簡化公式進行能力與難度參數的快速轉換¹¹：

1. 設測驗總題數為 L ，學生總人數為 N ¹¹。
2. 計算每位學生的原始得分 S_n （其中 $n = 1, \dots, N$ ）以及每道試題的答對人數 S_i （其中 $i = 1, \dots, L$ ）¹¹。
3. 計算學生原始得分之對數幾率（Log-odds）的樣本標準差 SD_N ，以及試題原始難度之對數幾率的樣本標準差 SD_L ¹¹。
4. 計算試題難度擴張因子 XL 與學生能力擴張因子 XN ¹¹：

$$XL = \sqrt{\frac{1 + SD_N/2.89}{1 - SD_L \cdot SD_N/8.35}}$$

$$XN = \sqrt{\frac{1 + SD_L/2.89}{1 - SD_L \cdot SD_N/8.35}}$$

5. 計算試題 i 的暫時難度： $D_i^{(0)} = -XL \cdot \ln\left(\frac{S_i}{N-S_i}\right)$ ¹¹。
6. 將試題難度進行中心化校正，使均值為零： $D_i = D_i^{(0)} - \bar{D}^{(0)}$ ¹¹。
7. 計算學生 n 的潛在能力值： $B_n = XN \cdot \ln\left(\frac{S_n}{L-S_n}\right) - \bar{D}^{(0)}$ ¹¹。
8. 估計兩者的模型標準誤（Standard Error）：

$$SE(D_i) = XL \cdot \sqrt{\frac{N}{S_i(N-S_i)}} \quad 11$$

$$SE(B_n) = XN \cdot \sqrt{\frac{L}{S_n(L-S_n)}} \quad 11$$

缺失數據或非對稱設計下的迭代 PROX 演算法

在日常小考中，常有學生因請假而缺考部分試題，此時必須採用迭代 PROX 演算法 ¹¹。設 N_i 為嘗試解答試題 i 的實際人數， μ_i 與 σ_i 分別表示這 N_i 位學生能力的均值與標準差 ¹¹：

- 試題難度迭代公式：

$$D_i = \mu_i - XL_i \cdot \ln\left(\frac{S_i}{N_i - S_i}\right)$$

¹¹

- 設 L_n 為學生 n 實際嘗試的試題數， μ_n 與 σ_n 分別表示這 L_n 道試題難度的均值與標準差 ¹¹：

$$B_n = \mu_n + XN_n \cdot \ln \left(\frac{S_n}{L_n - S_n} \right)$$

11

- 系統通過反覆更新學生與試題的參數，並在每次迭代後將難度均值歸零，直至最大參數變化量小於 0.01 Logit 時停止¹¹。這確保了在日常測驗存在零星缺考時，能力與難度的估計依然保持無偏與穩定¹¹。

此外，針對 polytomous 數據（如包含多級評分標準的作文、非選擇題手寫題），系統底層支援評級反應模型（Rating Scale Model, RSM）或部分得分模型（Partial Credit Model, PCM）⁹。這類模型要求每個分數階梯（Threshold）必須擁有至少 10 次以上的實際觀測值⁹，以確保階梯臨界難度估計的穩定度。若小樣本下觀測值不足，系統將自動啟動相鄰範疇合併機制，將多級評分合併為二分值進行穩健估計¹⁴。

確定性輸入與有雜訊的「與」閘模型（DINA）之微觀診斷

為了提供精細化的認知診斷反饋，系統在 App 內建構了 DINA 模型¹⁸。相較於 IRT 將學生能力視為單一連續變量，DINA 模型將學生的知識結構拆解為 K 個互相獨立的離散認知屬性（Cognitive Attributes），並將其表示為多維二進制向量 $\alpha_i = (\alpha_{i1}, \dots, \alpha_{iK})$ ，其中 $\alpha_{ik} \in \{0, 1\}$ 表示學生 i 對於屬性 k 的掌握狀態¹⁹。

Q 矩陣理論設計與驗證

DINA 模型的運作極度依賴由學科學術專家與導師共同編製的 $J \times K$ 二元矩陣——Q 矩陣（Q-matrix）²¹。矩陣中的元素 $q_{jk} \in \{0, 1\}$ 定義了答對第 j 題是否必須掌握第 k 個認知屬性²¹。

- 若 $q_{jk} = 1$ ，代表該試題對該技能有強制的依賴關係²¹。
- 若 $q_{jk} = 0$ ，則表示該技能並非解答該題的必要條件²¹。

在 App 的開發實務中，為了避免導師主觀編製 Q 矩陣所帶來的測量偏誤，系統整合了 Q 矩陣校正演算法¹⁸。透過分析學生實際作答數據的反應模式，計算項目殘差與不一致度，利用機器學習或相關係數檢定，自動偵測並提示可能存在定義不準確的 Q 矩陣單元，提示導師進行微調¹⁸。以下為數學科某單元測驗的 Q 矩陣設計範例：

試題編號 (j)	屬性 1: 基礎分數運算 (α1)	屬性 2: 通分與最簡分數轉換 (α2)	屬性 3: 代數方程邏輯推導 (α3)	理想反應模式代碼
試題 1	1	0	0	..
試題 2	1	1	0	..
試題 3	0	1	1	..
試題 4	1	1	1	..

DINA 數學形式與小樣本適應調優

DINA 模型的核心假設在於其「聯言性」 (Conjunctive Nature) ²²。若學生 i 意圖正確回答試題 j ，則其必須完全掌握該題所需的所有認知屬性，此時其理想反應狀態 η_{ij} 為 1，否則為 0 ²²：

$$\eta_{ij} = \prod_{k=1}^K \alpha_{ik}^{q_{jk}}$$

考慮到現實測驗中不可避免的隨機噪聲，系統引入了失誤參數 (Slipping, s_j) 與猜測參數 (Guessing, g_j) ¹⁹：

- 失誤參數 (s_j)：學生已掌握所有必要屬性，但因粗心或疲勞而答錯的概率 ¹⁹：

$$s_j = P(Y_{ij} = 0 \mid \eta_{ij} = 1)$$

- 猜測參數 (g_j)：學生未完全掌握必要屬性，但因隨機猜測而答對的概率 ¹⁹：

$$g_j = P(Y_{ij} = 1 \mid \eta_{ij} = 0)$$

此時，作答反應機率模型表達為：

$$P(Y_{ij} = 1 \mid \alpha_i) = g_j^{1-\eta_{ij}} (1 - s_j)^{\eta_{ij}}$$

在高中課堂日常評量（樣本量 $N \approx 35$ ）中，若採用飽和的貝氏估計，容易導致後驗分佈過於寬鬆¹⁹。本 App 的調優策略為：

- 將單次測驗分析的屬性個數限制在 $K \leq 5$ ²⁰。這能將潛在知識狀態空間限制在 $2^5 = 32$ 種模式內，大幅降低參數估計的難度¹⁹。
- 系統底層導入非參數分類法（Non-parametric Cognitive Classification, NPC）。該方法不需要對失誤與猜測參數進行大樣本機率估計，而是直接計算學生的實得分數向量與各知識狀態對應的理想作答向量之間的漢明距離（Hamming Distance），在極小樣本下依然能輸出極高信度的技能掌握剖面²⁴。

三、跨時間學術成長軌跡與增值評量模型之計量建構

單一時間點的考試成績只能提供靜態的「學術斷面」，無法反映教學過程對學生成長所帶來的實質影響³。為了解決這個痛點，系統內置了先進的縱向學術追蹤模型，透過排除起點行為、引入混和效應，並對長期增值軌跡進行多層建模，為導師提供科學的學術成長診斷³。

基於滯後分數（Lagged-Score）的增值評量模型（VAM）

增值評量模型（Value-Added Model, VAM）是現代教育評量中最具公平性與科學性的進步指標計算方法¹³。它承認不同學生的初始學術起點並不對等，拒絕使用簡單的「進步分數」（Gain Score，即期末分數減去期初分數，這會受到統計回歸效應的嚴重干擾），而是利用回歸分析，根據學生的歷史成績與背景變數，推算出其在本次測驗中應有的「預期表現」，並將實得成績與預期表現之差定義為增值殘差¹³。

VAM 線性模型之設定

假設追蹤班級學生 i 在學科 j 、第 t 次測驗（如期末考）的實得成績 Y_{ijt} ²⁷。回歸方程定義為²⁷：

$$Y_{ijt} = \beta_0 + \beta_1 Y_{ij,t-1} + \beta_2 Y_{ij,t-2} + \gamma \mathbf{X}_{it} + e_{ijt}$$

其中：

- $Y_{ij,t-1}$ 與 $Y_{ij,t-2}$ 分別為學生 i 在前一次與前二次測驗的歷史成績，作為控制初始能力的關鍵起點指標²⁶。
- $\mathbf{X}_{it}$ 為學生層次的控制變數向量（例如缺課天數、課堂參與度評級等）²⁶。
- $\beta_0, \beta_1, \beta_2, \gamma$ 為模型待估係數²⁷。
- e_{ijt} 為學生的個體殘差，即學生實際成績與基於同儕起點推估出之預期成績的差值，此即為該生的學業增值分數²⁶。

透過最小二乘法（OLS）擬合回歸方程後，學生 i 的實質學術增值 $\hat{e}_{ijt}$ 表達為²⁶：

$$\hat{e}_{ijt} = Y_{ijt} - \hat{Y}_{ijt} = Y_{ijt} - \left(\hat{\beta}_0 + \hat{\beta}_1 Y_{ij,t-1} + \hat{\beta}_2 Y_{ij,t-2} + \hat{\gamma} \mathbf{X}_{it} \right)$$

學生化殘差（Studentized Residuals）異常值檢定

為了對學生個體的增值表現進行科學的偏離度檢定，系統必須將原始殘差 $\hat{e}_{ijt}$ 轉換為無量綱的**學生化殘差（Studentized Residuals）**³³。這不僅消除了不同學科測驗總分尺度不一的影響，更能精確辨識出哪些進退步是由隨機誤差引起，哪些具備高度統計學顯著性³³。

第 i 位學生的學生化外在殘差 e_{ijt}^* 計算公式如下³³：

$$e_{ijt}^* = \frac{\hat{e}_{ijt}}{s_{-(i)} \sqrt{1 - h_{ii}}}$$

其中：

- $s_{-(i)}$ 為排除第 i 位學生數據後重新擬合模型所得之剩餘標準誤差³³。
- h_{ii} 為第 i 位學生在設計矩陣（Design Matrix）中的槓桿值（Leverage Value），用以反映該生初始起點特徵相較於全班均值的極端程度³³。

學生化殘差 e_{ijt}^* 嚴格服從自由度為 $N - p - 1$ 的 t 分佈（其中 p 為預測變量個數）³⁴。App 底層預設判定邏輯如下³⁴：

- 當 $e_{ijt}^* > 2.0$ 時，判定該生表現顯著優於預期，系統自動將其標記為「正偏離黑馬生」，提示導師其當期學習方法或外部輔導成效極佳¹³。
- 當 $e_{ijt}^* < -2.0$ 時，判定該生學術表現出現顯著崩跌，標記為「負偏離預警生」¹³。這能幫助導師精確捕捉到那些雖然絕對分數仍在及格線上、但成長軌跡已發生顯著退步的隱性危機生，從而發揮「預防重於治療」的教育防護網功能²⁶。

縱向多層線性模型（HLM）與潛在成長曲線分析（LGCA）

當一學期內累積了三次或以上的多次考試成績（ $T \geq 3$ ）時，App 的分析維度將自動升級為潛在成長曲線分析（Latent Growth Curve Analysis, LGCA），將學生的長期表現建構為多層線性模型（Hierarchical Linear Model, HLM），將時間點嵌套於學生個體內部，精準描繪其成長的動力學軌跡³⁵。

第一層模型（個體內部時間維度）

$$Y_{tij} = \pi_{0ij} + \pi_{1ij} \cdot (\text{Time}_t) + e_{tij}$$

36

其中：

- Y_{tij} 表示學生 i 在第 t 次考試、第 j 個學科的能力估計值³⁶。
- Time_t 為時間代碼（如 $t = 0, 1, 2, \dots$ ，代表各次測驗的時間間距）³⁶。
- π_{0ij} 為學生 i 在該學科的初始潛在學術水平（截距）³⁶。
- π_{1ij} 為學生 i 隨時間演進的學術進步速率（斜率）³⁶。
- e_{tij} 為第一層測量殘差³⁶。

第二層模型（個體間差異維度）

系統進一步將學生的初始水平與成長速率視為隨機變量，建立第二層模型³⁶：

$$\pi_{0ij} = \gamma_{00j} + r_{0ij}$$

$$\pi_{1ij} = \gamma_{10j} + r_{1ij}$$

36

其中：

- γ_{00j} 與 γ_{10j} 分別代表全班學生在學科 j 的平均初始起點與平均進步速率（固定效應）³⁶。
- r_{0ij} 與 r_{1ij} 分別為學生個體的截距與斜率之隨機變異（隨機效應）³⁶。

系統會特別去計算 r_{0ij} 與 r_{1ij} 之間的協方差 σ_{01j} ，這具備深度的教學診斷價值³⁵：

- 若協方差呈顯著正相關，代表「起點高的人進步愈快」，班級內的學術分化（雙峰效應）正逐漸擴大¹²。
- 若呈顯著負相關，則代表「起點低的人進步較快」，提示班級內部的低分群學生正在快速追趕，班級學術同質性逐步提升，導師採用的均等化教學成效顯著¹³。

在小樣本分析中，App 更引入了**混合效應位置量尺模型（Mixed-Effect Location Scale Model, MELS）**³⁷。傳統模型假設個體內殘差方差 e_{tij} 為常數，而 MELS 模型允許殘差方差隨學生起點或時間變量而改變³⁷。這有助於診斷班級中哪些學生存在「高波動學術表現」（即成績起伏巨大），這類學生的學術穩定度低，往往伴隨心理波動或學習習慣不良，是導師需重點關懷的對象³⁷。

四、App 統計圖表模組之設計規格與前端分析演算法

為使上述複雜的教育計量與統計分析方法能具體落地，本節設計了高階統計圖表的 UI 設計規格，並提供了一套可直接在 App 前端（本地端）快速運行、免除伺服器依賴的 JavaScript 統計計算引擎⁴。

統計診斷圖表模組之 UI 設計規格

App 的視覺化界面應明確區分為「班級集體」與「學生個體」雙端維度，各圖表規格設計如下表所示：

分析層次	核心圖表名稱	計量理論基礎	具體 UI 呈現與動態交互設計規格	教育決策與實務診斷價值
班級集體 (Class Level)	1. CTT 項目診斷二維散佈圖	經典測驗理論 項目分析 ⁴	橫軸表示難度 (P_j)，縱軸表	自動偵測並剔除不具鑑別度的瑕疵試題，

			示鑑別度 (r_{pbi})。7 象限圖設計，理想項目落入第一象限。點擊試題點可動態展開誘答選項柱狀圖，提示選擇率低於 5% 的非功能性選項⁴⁰。	幫助學科教師優化題庫品質，維護未來測驗的信度 ⁷ 。
	2. 跨學科真實能力關聯矩陣熱圖	信度衰減校正 Pearson 相關矩陣	熱圖單元格呈圓形，大小與顏色深淺對應校正後的真實能力相關係數 $r_{T_x T_y}$。點擊單元格可動態繪製兩學科 Logit 能力值的二維置信橢圓與回歸線¹。	診斷班級學科間的共變結構，探查是否存在跨學科能力的系統性制約（如數學不佳直接限制了物理進步），指導跨科教學對策²¹。
	3. 班級 longitudinal 成長扇形帶	HLM 潛在成長曲線固定與隨機效應³⁶	橫軸為測驗次數，縱軸為等值化後的 Logit 能力。中央實線表示班級平均成長路徑 γ_{10}，周圍漸變藍色扇形帶代表一個標準差範圍³⁶。	評估班級整體學術高度是否隨時間實質攀升，以及學生成績分佈是否趨向均等（扇形帶收窄）或分化（扇形帶擴大）¹²。
學生個體	4. DINA 知識屬	DINA 潛在掌握概率 α_{ik} 估計	雷達圖頂點代表各項微觀學	擺脫單一總分的粗糙評定，

(Student Level)	性診斷雷達圖	20	科技能（如因式分解、幾何邏輯）。陰影面積代表各屬性掌握概率。點擊頂點可呼叫對應之錯題集與補救教材資源 ²⁰ 。	直接向學生與家長揭示細粒度的認知盲區，提供精確補救教學路徑 ²¹ 。
	5. 個體 VAM 成長軌跡診斷圖	學生化外在殘差與預測折線 ³³	實線表示學生各次考試的實際能力，虛線表示 VAM 模型推估出的預期表現 $\hat{Y}_{it}$ 。 ²⁶ 當點擊某個時間點時，若學生化殘差極端，點會呈現綠色（正偏離）或紅色（負偏離警告） ²⁶ 。	精確捕捉「高起點、低增值」的隱性退步學生，以及「低起點、高增值」的逆襲進步生，落實精準關懷與預警 ¹³ 。

核心統計分析引擎 JavaScript 實作代碼

以下為 App 前端開發的核心統計類別，以高度優化的矩陣運算，完整實現 CTT 誘答分析、Rasch PROX 估計，以及 VAM 學生化殘差計算¹¹：

```
JavaScript
/**
 * 班級與個體多維教育統計診斷引擎 (Advanced Psychometric Engine)
 */
class PsychometricEngine
```

```

/**
 * 1. 經典測驗理論 (CTT) 與誘答選項效率分析
 * @param {Array<Array<string>>} responses - 學生原始作答二維陣列 (N 人 x J 題, 內容為選取的
選項如 "A", "B" 等)
 * @param {Array<string>} keys - 試卷標準答案 (長度為 J 的一維陣列)
 * @returns {Object} CTT 指標與誘答效率報告
 */

static analyzeCTTAndDistractors(responses, keys)
    const N = responses.length
    const J = keys.length
    if (N === 0 || J === 0) throw new Error "無效的作答數據矩陣"

    // 1.1 計算每位學生的原始總分
    const studentScores = new Array(N).fill(0)
    const studentProfiles =

    for (let n = 0; n < N; n++) {
        let score = 0
        for (let j = 0; j < J; j++) {
            if (responses[n][j] === keys[j]) score++
        }
        studentScores[n] = score
        studentProfiles.push({ id: n, score: score, rawResponses: responses[n] })
    }

    // 1.2 進行極端組劃分 (高低分組各取 27%)
    studentProfiles.sort((a, b) => b.score - a.score)
    const extremeSize = Math.max(1, Math.round(N * 0.27))
    const highGroup = studentProfiles.slice(0, extremeSize)
    const lowGroup = studentProfiles.slice(-extremeSize)

    const itemReports =

    // 1.3 逐題進行難度、極端組鑑別度與選項分析
    for (let j = 0; j < J; j++) {
        let correctCount = 0
        const selectionCounts = []

        for (let n = 0; n < N; n++) {
            const choice = responses[n][j]

```

```

        selectionCounts[choice] = (selectionCounts[choice] || 0) + 1
        if (choice === keys[j]) correctCount++
    }

    const pValue = correctCount / N // 難度 (Item Difficulty, CTT)

    // 計算極端組鑑別度 (D)
    const highCorrect = highGroup.filter(s => s.rawResponses[j] === keys[j]).length
    const lowCorrect = lowGroup.filter(s => s.rawResponses[j] === keys[j]).length
    const dIndex = (highCorrect / extremeSize) - (lowCorrect / extremeSize)

    // 誘答選項效率診斷
    const optionDiagnostics =
    const presentOptions = Object.keys(selectionCounts)

    presentOptions.forEach(opt => {
        const rate = selectionCounts[opt] / N
        const isCorrect = (opt === keys[j])
        // 有效誘答標準：非正解、且至少有 5% 的學生選擇
        const isFunctionalDistractor = !isCorrect && rate >= 0.05

        optionDiagnostics.push({
            option: opt,
            frequency: selectionCounts[opt],
            selectionRate: rate,
            isCorrect: isCorrect,
            isFunctionalDistractor: isFunctionalDistractor
        })
    })

    // 判定該項目之計量學狀態
    let recommendation = "保留 (Retain)"
    if (pValue < 0.20 || pValue > 0.85) recommendation = "難度偏激，建議修訂 (Modify Difficulty)"
    if (dIndex < 0.20) recommendation = "鑑別度不足，建議剔除或大修 (Review Item)"

    itemReports.push({
        itemId: item,
        difficulty: pValue,
        discrimination: dIndex,
        options: optionDiagnostics
    })

```

```

        status recommendation
    });
}

return [ItemReports, studentScores];
}

/**
 * 2. Rasch 單參數模型之正常近似估計 (PROX 演算法)
 * @param {Array<Array<number>>} binaryMatrix - 學生答題二元對錯矩陣 (N 人 x L 題, 答對為
1, 答錯為 0)
 * @returns {Object} 估計之能力、難度、標準誤與適配指標
 */
static estimateRaschPROX(binaryMatrix) {
    const N = binaryMatrix.length;
    const L = binaryMatrix[0].length;

    const S_n = new Array(N).fill(0) // 學生得分向量
    const S_i = new Array(L).fill(0) // 試題答對數向量

    for (let n = 0; n < N; n++) {
        for (let i = 0; i < L; i++) {
            if (binaryMatrix[n][i] === 1) {
                S_n[n]++;
                S_i[i]++;
            }
        }
    }

    // 極端分數校正 (極端值不能直接取自然對數, 否則會產生無限大或無限小)
    const adjustExtremes = (score, max) => {
        if (score === 0) return 0.25;
        if (score === max) return max - 0.25;
        return score;
    };

    // 2.1 計算初始 logit 值
    const personLogits = S_n.map(r => Math.log(adjustExtremes(r, L) / (L -
adjustExtremes(r, L))));
    const itemLogits = S_i.map(s => Math.log((N - adjustExtremes(s, N)) / adjustExtremes(s,
N)));

```



```

// 輔助統計函數
const calcMean = arr => arr.reduce((a, b) => a + b, 0) / arr.length
const calcStdDev = (arr, m) => Math.sqrt(arr.map(x => Math.pow(x - m, 2)).reduce((a, b)
=> a + b, 0) / (arr.length - 1))

const meanN = calcMean(personLogits)
const meanL = calcMean(itemLogits)
const SD_N = calcStdDev(personLogits, meanN)
const SD_L = calcStdDev(itemLogits, meanL)

// 2.2 計算校正因子 (Expansion Factors)
const scaleConstant = 1.0 / (SD_L * SD_N) * 8.35
const XL = Math.sqrt(1.0 + SD_N * 2.89 / scaleConstant)
const XN = Math.sqrt(1.0 + SD_L * 2.89 / scaleConstant)

// 2.3 計算最終試題難度並進行中心化校正
let tempDiff = S_i.map(s => XL * Math.log(adjustExtremes(s, N) / (N - adjustExtremes(s,
N))))
const meanTempDiff = calcMean(tempDiff)
const difficulties = tempDiff.map(d => d - meanTempDiff)

// 2.4 計算最終學生潛在能力 (均值調整對齊)
const abilities = S_n.map(r => XN * Math.log(adjustExtremes(r, L) / (L - adjustExtremes(r,
L))) - meanTempDiff)

// 2.5 計算估計誤差 (Standard Errors)
const itemSE = S_i.map(s =>
  const adjS = adjustExtremes(s, N)
  return XL * Math.sqrt(N / (adjS * (N - adjS)))
)
const personSE = S_n.map(r =>
  const adjR = adjustExtremes(r, L)
  return XN * Math.sqrt(L / (adjR * (L - adjR)))
)

return [abilities, difficulties, personSE, itemSE]

/**
 * 3. 增值模型 (VAM) 滯後回歸與學生化殘差檢定

```

```

* @param {Array<number>} prior1 - 基準考試 1 之等值化能力值 (Y_t-1)
* @param {Array<number>} prior2 - 基準考試 2 之等值化能力值 (Y_t-2)
* @param {Array<number>} current - 本次測驗之等值化能力值 (Y_t)
* @param {Array<number>} covariates - 學生層次控制變數 (如缺課天數)
* @returns {Object} VAM 參數、預期能力、學生化殘差與異常診斷
*/
static calculateVAM(prior1, prior2, current, covariates)
  const N = current.length
  if (N <= 4) throw new Error "數據樣本量過小，無法進行回歸分析"

  // 3.1 構建設計矩陣 X 與因變量向量 Y (OLS 多元線性回歸)
  // 模型: Current = b0 + b1*Prior1 + b2*Prior2 + b3*Covariate
  const X =
  const Y = [_, current]

  for (let i = 0; i < N; i++)
    X.push([1, prior1[i], prior2[i], covariates[i]])

  // 矩陣乘法與轉置輔助函數
  const transpose = A => A.map( (_, colIdx) => A.map( row => row[colIdx]) )
  const multiply = (A, B) =>
    const result = new Array(A.length).fill(0).map( () => new Array(B.length).fill(0) )
    return result.map( (row, i) => row.map( (_, j) => A[i].reduce( (sum, val, k) => sum + val * B[k][j], 0 ) ) )

  // 簡易 4x4 矩陣求逆 (高斯-若爾當消去法)
  const invert4x4 = A =>
    const n = 4
    const C = A.map( (row, i) => [_, row, ...new Array(n).fill(0).map( (_, j) => i === j ? 1 : 0 ) ] )
    for (let i = 0; i < n; i++)
      let maxRow = i
      for (let k = i + 1; k < n; k++)
        if (Math.abs(C[k][i]) > Math.abs(C[i][i])) maxRow = k
      const temp = C[i]; C[i] = C[maxRow]; C[maxRow] = temp
      if (Math.abs(C[i][i]) < 1e-12) return null // 奇異矩陣
      for (let k = i + 1; k < 2 * n; k++) C[i][k] /= C[i][i]
      C[i][i] = 1

```

```

    for (let k = 0; k < n; k++) {
      if (k !== i) {
        const factor = C[k][i];
        for (let j = i; j < 2 * n; j++) C[k][j] -= factor * C[i][j];
      }
    }
  }
  return C.map(row => row.slice(n));
}

```

```

const Xt = transpose(X);
const XtX = multiply(Xt, X);
const XtX_inv = invert4x4(XtX);

```

```

if (!XtX_inv) {
  throw new Error("設計矩陣共線性極高，無法逆矩陣求得不偏估計值");
}

```

```

const Y_col = Y.map(y => [y]);
const XtY = multiply(Xt, Y_col);
const B_col = multiply(XtX_inv, XtY);
const coefficients = B_col.map(row => row); // [b0, b1, b2, b3]

```

```

// 3.2 計算預估值與原始殘差
const expected = [];
const residuals = [];
let sumSquaredResiduals = 0;

```

```

for (let i = 0; i < N; i++) {
  const pred = coefficients[0] +
    coefficients[1] *
    coefficients[2] * prior2[i] +
    coefficients[3] * covariates[i];
  const res = current[i] - pred;
  expected.push(pred);
  residuals.push(res);
  sumSquaredResiduals += res * res;
}

```

```

const df = N - 4 // 自由度 (N - 參數個數)

```

```

const residualVariance = sumSquaredResiduals / df

// 3.3 計算槓桿值 ( $H_{ii} = \text{diag}(X(XtX)^{-1}Xt)$ )
const H = multiply(multiply(X, XtX_inv), Xt);
const studentizedResiduals =
const diagnostics =

// 3.4 計算學生化外在殘差 (Externally Studentized Residual)
for let i = 0; i < N; i++) {
  const h_ii = H[i][i];

  // 排除第 i 筆數據之殘差方差估計
  const numerator = sumSquaredResiduals - (residuals[i] * residuals[i]) / (1 - h_ii);
  const s_minus_i_sq = Math.max(0.001, numerator / (df - 1));
  const s_minus_i = Math.sqrt(s_minus_i_sq);

  const studentizedRes = residuals[i] / (s_minus_i * Math.sqrt(1 - h_ii));
  studentizedResiduals.push(studentizedRes);

  // 異常閾值判定
  let status = "穩健成長 (Stable Progress)";
  if (studentizedRes > 1.96) status = "超預期成長黑馬 (High Progress Outlier)";
  if (studentizedRes < -1.96) status = "顯著下滑預警 (Critical Decline Warning)";

  diagnostics.push({
    studentId,
    actualValue: current[i],
    expectedValue: expected[i],
    residual: residuals[i],
    studentizedResidual: studentizedRes,
    diagnosticStatus: status
  });
}

return [coefficients, diagnostics, residualVariance];

```

五、結論與應用開發建議

為了建構一套具備學術嚴謹性、能有效輔助實務教學，且兼顧日常班級經營便利性的「雙核心成

績統計與診斷 App」，本報告彙整了以下三項具體的開發設計指引：

精準平衡學術嚴謹度與本地端運算效能

高中的成績診斷分析必須同時面對「運算效率」與「統計穩定度」的雙重挑戰⁴。

- 系統內置的 **Rasch 估計引擎（PROX 類）** 與 **VAM 滯後回歸模組**，應完全基於前述優化過的輕量化數學閉式公式編寫¹¹。
- 這使得所有的矩陣分解、對數幾率（Logit）轉換與標準誤校正¹¹，均能在學生端或導師端的手機、平板本地端（Edge Computing）即時運算完畢⁴，免除將班級成績上傳至雲端大型伺服器所帶來的資安與隱私風險。

以多層維度重塑成績報告架構

未來的 App 應徹底屏棄僅呈現單一原始總分（如 65 分）的簡陋傳統報告⁴，改為提供以下三維度的資訊：

- **能力定位（Rasch Logit）**：用以衡量學生不受考試難易度影響的真正學術高度⁹。
- **細粒度認知掌握度（DINA Skill Profile）**：精確呈現在單元知識網絡中，學生究竟是哪些具體概念尚未學會，藉此提供全自動的個性化精準補救教學單²⁰。
- **進步增值指標（VAM Studentized Residual）**：用以客觀呈現學生自我超越的幅度，使教師能公平地看見每位學生的努力與實質進步¹³。

實踐「預防重於補救」的教育防護網

透過將**多層縱向成長曲線（LGCA）**與**學生化殘差異常值檢定**深度融入日常 App 的分析模組中，導師得以不再扮演「事後補救」的被動角色³⁴。

- 系統的背景演算引擎能在每次段考結束後，自動對全班進行掃描，在極早期便揪出那些**絕對分數雖然維持在及格線上、但「增值殘差」已連續呈顯著負偏離（即學術表現顯著低於預估軌跡）的隱性退步學生**²⁶。
- 這讓導師能搶在學生信心完全崩潰、學科徹底落後之前，精確判別其知識盲區並及時介入輔導²¹，將教育計量學的統計智慧，轉化為守護學生學術發展最具溫度的實踐路徑。